

COMPTE-RENDU – PROJET

INFORMATIQUE ELECTRONIQUE

Projet – S6 – EI2I - 3ème année



TALHAOUI Sofian & KHERCHAOUI Samir

À l'attention de Mr Rémi Griot.

Edition 2016-2017

Table des matières

Table des matières.....	2
I. Introduction.....	2
II. Partie Informatique.....	3
A. ASM.....	5
B. AST.....	8
C. BISON.....	9
D. GRAF.....	9
E. GRAFINTERPRETER.....	9
F. LEX.....	9
G. VM.....	10
H. BUILD.....	10
I. MAKEFILE.....	10
III. Partie Electronique.....	15

I. Introduction

Dans le cadre de notre formation du Diplôme Ingénieur Electronique Informatique parcours Informatique Industrielle (EI-2I), nous devons effectuer un projet informatique électronique.

Ce rapport est un compte rendu de ce projet informatique-électronique.

II. Partie Informatique

Dans cette partie, l'objectif est de réaliser un compilateur en langage C. Ce projet est ségrégé en plusieurs parties.

- le compilateur qui traduit le fichier .grafcet en fichier .asm
- l'assembleur récupère le fichier.asm et génère le fichier .bin
- la machine virtuelle qui exécute le fichier .bin

Le compilateur interprète le grafcet (analyse lexicale → Lex, analyse syntaxique → bison, construction et exploitation d'un arbre syntaxique → ast).

L'assembleur contient le fichier asm.c et asm.h qui servent pour faire l'exécutable asm.out, cet exécutable génère le fichier .asm.

L'assembleur contient le fichier asm.c et asm.h qui servent pour faire l'exécutable asm.out, cet exécutable génère le fichier .bin.

On a d'abord commencé par traduire à la main un grafcet en C puis nous l'avons traduit en assembleur et enfin en binaire.

Nous avons ensuite codé la machine virtuelle qui exécute le fichier binaire. Une fonction run() indique ce que font les différentes instructions : ADD additionne les deux derniers nombres de la pile par exemple. Dans le fichier binaire, les instructions sont représentées par des nombres. Ils sont définis dans le fichier header vm_codeops.h. Une fonction readBin, qui prend en arguments un fichier binaire et un tableau de code, va afficher sur la console le nombre de lignes du fichier et les instructions.

Cette machine virtuelle devait ensuite être exportée sur micro-contôleur dans le cadre du projet électronique.

Ensuite, nous avons commencé à coder l'assembleur (fichier asm.c) ainsi que le fichier header asm.h. La première fonction codée était la fonction parseAsm qui prend un fichier en paramètre.

Elle détecte si la ligne est un commentaire (commence par #) ou une étiquette (contient " : ") ou enfin une instruction.

Si c'est un commentaire, la fonction passe à la ligne suivante. Si c'est une étiquette, elle fait appel à la fonction addLabel qui complète le tableau de structures contenant les labels et incrémente le compteur de labels.

Dans le cas d'une instruction, elle fait appel à une fonction decodeInstruction() qui, comme son nom l'indique, décodera l'instruction. Cette fonction fait le gros du travail dans l'assembleur, elle identifie le type de l'instruction en la comparant à un dictionnaire d'instructions qui nous a été donné. Elle fait aussi le décodage

supplémentaire, elle appelle également une autre fonction, `addCode`, pour remplir `codeSegment` (qui correspond au tableau de code dans la VM). Pour finir, si l'instruction décodée fait référence à une étiquette encore inconnue, elle ajoute la référence à un tableau de structure de références.

Les instructions peuvent être de 3 types :

- type 0 : elles ne demandent pas de décodage d'opérandes supplémentaires (0 argument)

```
addInstructionName("add", OP_ADD, 0, "", 0);
addInstructionName("sub", OP_SUB, 0, "", 0);
addInstructionName("mult", OP_MULT, 0, "", 0);
addInstructionName("neg", OP_NEG, 0, "", 0);
addInstructionName("and", OP_AND, 0, "", 0);
addInstructionName("or", OP_OR, 0, "", 0);
addInstructionName("not", OP_NOT, 0, "", 0);
addInstructionName("eq", OP_EQ, 0, "", 0);
addInstructionName("ls", OP_LS, 0, "", 0);
addInstructionName("gt", OP_GT, 0, "", 0);
addInstructionName("halt", OP_HALT, 0, "", 0);
```

- type 1 : le décodage d'un entier est nécessaire. Il y a une opérande (1 argument)

```
addInstructionName("push", OP_PUSH, 1, "%s %d", 1);
addInstructionName("pop", OP_POP, 1, "%s %d", 1);
addInstructionName("pushi", OP_PUSHI, 1, "%s %d", 1);
```

- type 2 : le décodage d'une chaîne de caractères représentant une étiquette est nécessaire. Il y a une opérande (1 argument)

```
addInstructionName("jp", OP_JP, 3, "%s %s", 1);
addInstructionName("jf", OP_JF, 3, "%s %s", 1);
```

On a ensuite codé la fonction `dumpBinaryCode()` qui est une fonction de vérification qui désassemble le code. Elle permet de générer le code assembleur correspondant aux `OP_CODE` stockés dans `codeSegment`. Le code s'affiche normalement en console mais nous avons un bug avec cette fonction que nous n'avons pas réussi à résoudre.

Nous avons ensuite les fonctions `resolveReferences()` qui résout les références liées aux étiquettes inconnues et la fonction `generateBinary` qui prend comme argument le fichier binaire qui sert à écrire le code binaire dans ce fichier.

Pour le compilateur, on devait compléter le fichier `ast.c`, le fichier `ast.h` était déjà complet. Nous devons remplir les fonctions `makeDual` et `generateCode`. Il fallait étudier toutes les fonctions présentes dans `ast.c` pour pouvoir compléter ces fonctions, ce qui n'était pas évident. Pour pouvoir passer à la suite et enchaîner sur la partie électronique, nous nous sommes aidées du compilateur fourni par Mr PECHEUX pour compléter ces fonctions.

A. ASM

Asm.c : Assembleur fait (`//addInstructionName("div", I_DIV, 0, "", 0)`; pas utilisé)

TYPE 1 : `//le %s -> l'instruction et le %d -> l'opérande`

TYPE 3 : `//le %s -> l'instruction et le 2ème %s -> l'étiquette`

```
/*      for(i=0;i<15;i++){ printf("%s\n",tabInstructionNames[i].name);  
printf("%d\n",tabInstructionNames[i].opcode);  
printf("%d\n",tabInstructionNames[i].type);  
printf("%d\n",tabInstructionNames[i].nbops);    printf("\n");}*/
```

Ce programme permet d'assembler un fichier assembleur (`safe.asm`) en bytecode dans un fichier `safe.bin`

Il fonctionne en deux parties :

- 1ère phase d'assemblage :
 - Génère un tableau de `OP_CODE`
 - Analyse et décode les instructions
 - Construit la liste des références indéfinies
- 2ème phase d'assemblage :
 - Complète le tableau de `OP_CODE`
 - Résous les références indéfinies

Liste des fonctions:

* - void **addInstructionName**(char *name, int opcode, int type, char *format, int nbops)

La fonction `addInstructionName(char *name, int opcode, int type, char *format, int nbops)` permet la construction d'un dictionnaire d'instruction, on ajoute alors à partir du main toutes les instructions que l'on utilisera en assembleur.

Elles peuvent être de 3 types :

- type 0 : elles ne demandent pas de décodage d'opérandes supplémentaires (0 argument)
- type 1 : le décodage d'un entier est nécessaire. Il y a une opérande (1 argument)
- type 3 : le décodage d'une chaîne de caractères représentant une étiquette est nécessaire. Il y a une opérande (1 argument)

* - void **addCode**(char *line, int opcod, char *format, int type) : Complète le tableau codeSegment

La fonction addCode(char *line, int opcod, char *format, int type) est appelée après le décodage de l'instruction. Son type a été déterminé, on peut donc récupérer l'instruction décodée et son opérande puis le placer dans le tableau d'opcode. Cette fonction fait le gros du travail de la phase 1, en identifiant le type de l'instruction, en faisant le décodage supplémentaire, et en ajoutant des références.

* - void **decodeInstruction**(char *line)

La fonction decodeInstruction(char *line) est appelée par la boucle principale d'analyse lorsque la ligne d'assembleur lue n'est pas une étiquette (pas de : dans la ligne) ni un commentaire (pas de # dans la ligne). Elle fait appel à addCode() pour remplir codeSegment lorsque l'instruction est identifiée.

La fonction decodeInstruction(char *line) est appelée par la boucle principale d'analyse lorsque la ligne d'assembleur lue n'est pas une (pas de : dans la ligne). Cette fonction fait le gros du travail de la phase 1, en identifiant le type de l'instruction, en faisant le décodage supplémentaire, en appelant addCode() pour remplir codeSegment et en ajoutant des références si l'instruction décodée fait référence à une étiquette encore inconnue

* - void **resolveReferences**(void)

La fonction resolveReferences(void) est appelée après l'analyse de l'assembleur afin de remplacer les références non résolues de l'opcode par l'adresse de l'opcode à laquelle elle renvoie

```
pop    23
pushi  0
pop    27
pushi  0
pop    24
pushi  0
```

pop 28

pushii. Pour cela on utilise le tableau des références et le tableau des labels.

* - void **addLabel**(char labelname[MAX_LINE_SIZE],int addr)

La fonction addLabel(char labelname[MAX_LINE_SIZE],int addr) est appelée lorsque qu'une étiquette est identifié lors de l'analyse de l'assembleur. Elle permet de déterminer l'étiquette et de la placer dans le tableau des labels

* - void **parseAsm**(FILE *fin)

Cette fonction parseAsm(FILE *fin) parcourt les lignes du fichier source assembleur tant qu'elle n'a pas rencontre de ligne avec "halt" à l'interieur. Pour chaque ligne il y a trois cas de figure:

- si la ligne de texte contient un #, il s'agit d'un commentaire alors on passe à la ligne suivante
- si la ligne de texte contient un :, il s'agit d'une etiquette et on la declare
- sinon, il s'agit d'une instruction et on appelle decodeInstruction()

* - void **generateBinary**(FILE *fout)

Cette fonction generateBinary(FILE *fout) permet de générer le fichier .bin qui contient l'opcode décodé ainsi que le nombre d'instruction.

C'est la fonction principale du programme qui va demander l'analyse du fichier.asm puis il va demander la résolution des références et enfin la création du fichier.bin

Asm.h

Ce programme est directement rattaché à asm.c.

On y retrouve les prototypes des fonctions utilisées ainsi que les structures des instructions.

- structure d'une étiquette
- structure d'une référence
- structure des instructions
- Prototypes des fonctions

B. AST

Ast.c

Constructeur et interpréteur de l'arbre syntaxique (complété)

Ce programme est un compilateur, il travaille directement avec bison, lex et ast.h.

Il nous a été fourni en grande partie pour la réalisation de notre projet.

Il permet, à partir d'un grafct de générer un code assembleur qui l'interprète.

Les fonctions maakeDual et generateCode sont appelés depuis y.tab.c

*

génératecode & makedual

while (transition!=NULL)

Si étape trouvé

on peut affecter le pointeur vers l'etape a l'etapeRefNode en cours

on peut ajouter la transition courante a la liste des transitions aval de l'etape que l'on vient d'identifier auquel cas on empile une nouvelle RefTransition dans la liste des transitions aval de l'etape

// on va parcourir la liste des etapes aval de la transition, qui

// est construite sous la forme d'une liste de etapeRefNode

// dont seule la partie name et le pointeur next sont pour l'instant remplis avec

// des informations

while (ern!=NULL)

// si on trouve l'etape, alors cela signifie deux choses

// 1) que l'on peut

// affecter le pointeur vers l'etape a l'etapeRefNode en cours

// 2) que l'on peut ajouter la transition courante a la liste

// des transitions amont de l'etape que l'on vient d'identifier

// auquel cas on empile une nouvelle RefTransition dans

// la liste des transitions amont de l'etape


```
// dernière passe, on parcourt les étapes, et en particulier leurs listes  
// de transitions amont et aval, de manière à modifier le champs transition  
// des éléments transitionRefNode pour mettre la structure de données d'équerre
```

Ast.h

C. BISON

grafcet.y

y.dot

y.tab.c

y.tab.h

y.tab.h.gch

D. GRAF

basic2.grafcet

basic.grafcet

ex1.grafcet

ex2.grafcet

safe.grafcet

E. GRAFINTERPRETER

graphInterpreter .c

Graphe interpreter

#1 Lecture des entrées #2 Calcul des fronts #3 Tant que stable==0

#4 Initialisation de appel_ex et reponse_ex #5 Initialisation de old_ex

#6 Sauvegarde des états dans old_ex #7 Calcul de appel_ex et reponse_ex

#8 Calcul des ex #8 Calcul de la stabilité Fin Tant que

F. LEX

grafcet.l**lex.yy.c**

G. VM

evm.c**vm_codops.h**

H. BUILD

Est le fichier permettant de tout compiler et lancer d'un seul trait dans le terminal

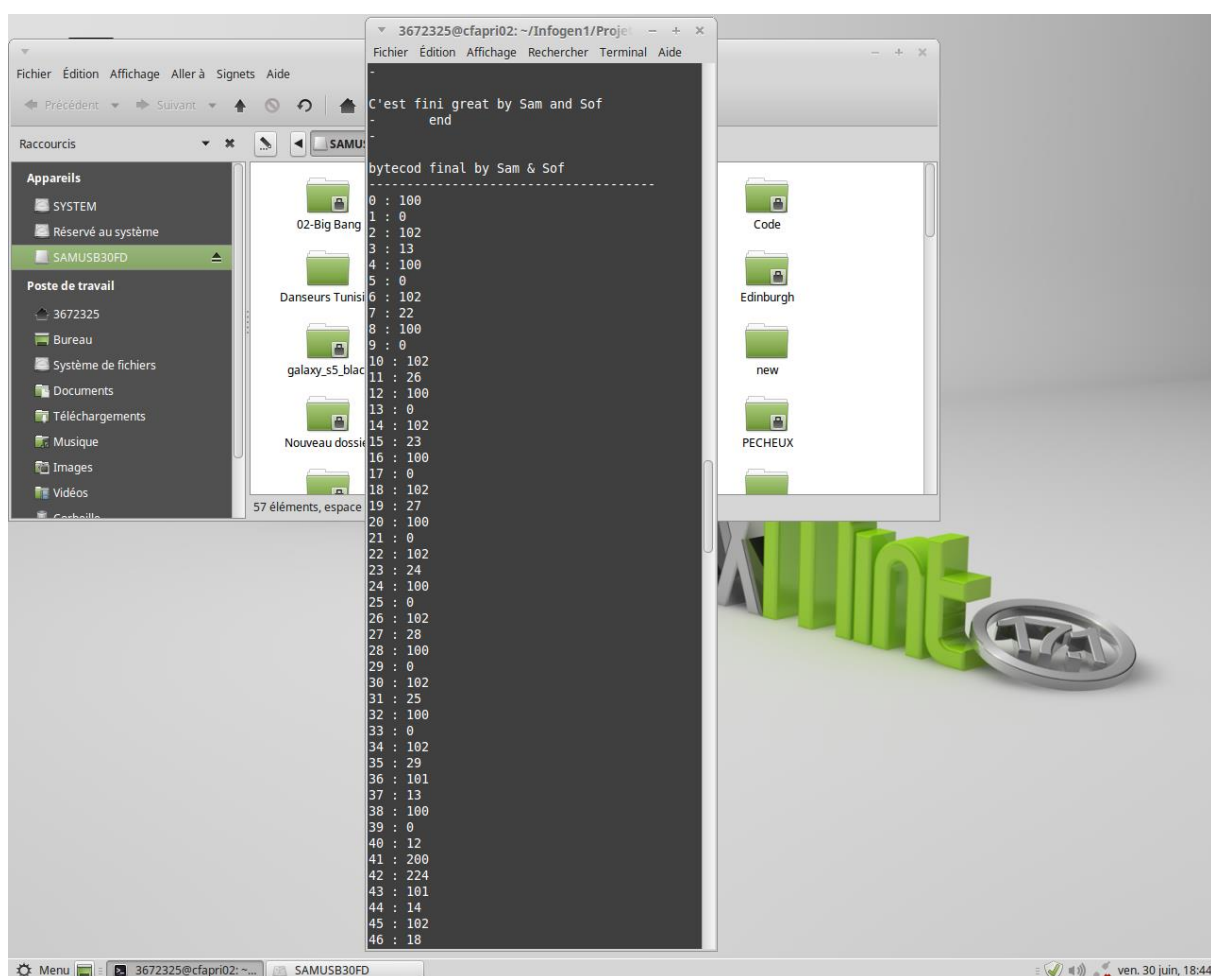
```
1  #!/bin/bash
2  cd ./bison
3  bison -y -d -g ./grafcet.y
4  cd ../lex
5  flex ./grafcet.l
6  cd ..
7
8  # Creation du compilateur de grafcet
9  make grafcet.out
10
11 # Compilation (.grafcet vers .asm)
12 ./grafcet.out < ./graf/safe.grafcet > safe.asm
13
14 # Creation de l'assembleur
15 make asm.out
16
17 # Assemblage (asm -> bytecode)
18 ./asm.out safe.asm safe.bin
19
20 #Clean
21 make clean
```

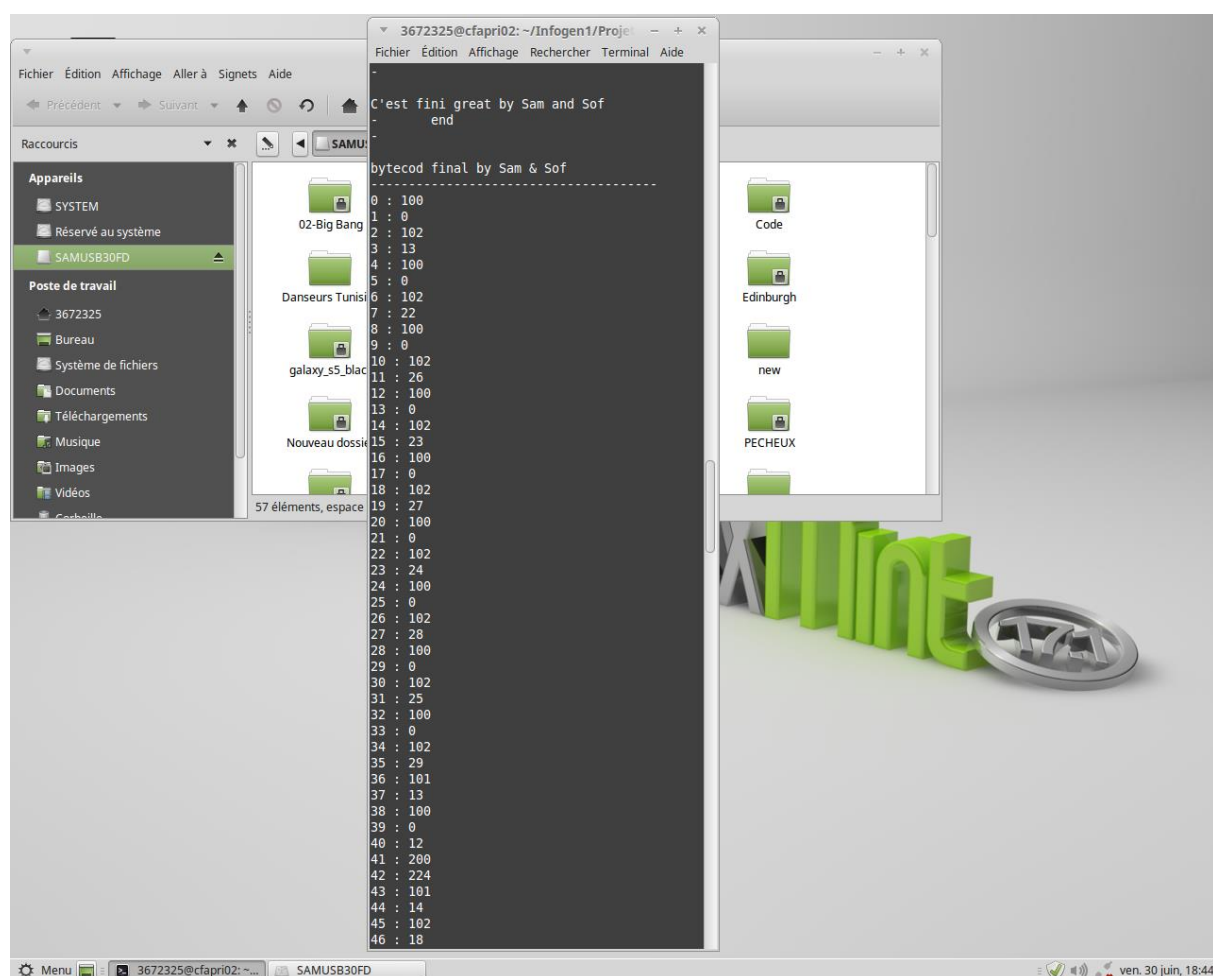
I. MAKEFILE

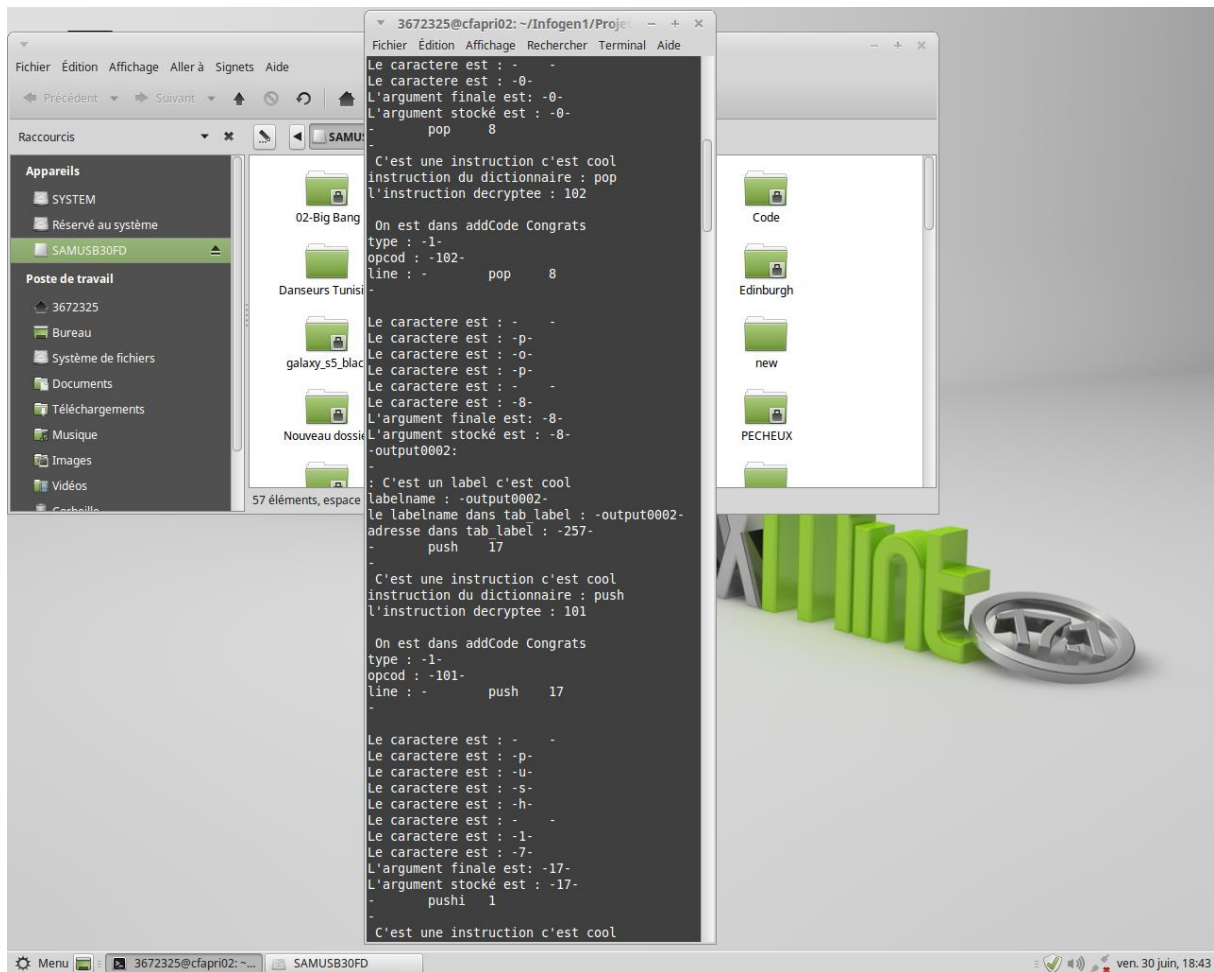
Le makefile général appelé dans le fichier ci-dessus.

```
1  # Var
2  IH = -I./ast
3
4  ##### Grafcet #####
5  grafcet.out : y.tab.o lex.yy.o ast.o
6      gcc y.tab.o lex.yy.o ast.o -o $@
7
8  ast.o : ./ast/ast.c ./ast/ast.h
9      gcc -c ./ast/ast.c $(IH)
10
11  y.tab.o : ./bison/y.tab.c ./bison/y.tab.h ./ast/ast.h
12      gcc -c ./bison/y.tab.c -I./bison $(IH)
13
14  lex.yy.o : ./lex/lex.yy.c ./ast/ast.h ./bison/y.tab.h
15      gcc -c ./lex/lex.yy.c -I./bison $(IH)
16
17  ##### Assembleur #####
18  asm.out : ./asm/asm.c ./asm/asm.h ./vm/
19      gcc ./asm/asm.c -I./asm/ -I./vm -o $@
20
21  cleano :
22      rm -f *.o
23  cleanout :
24      rm -f *.out
```

Voici notre fichier bien compiler on affiche bien ce qu'on génère.







III. Partie Electronique

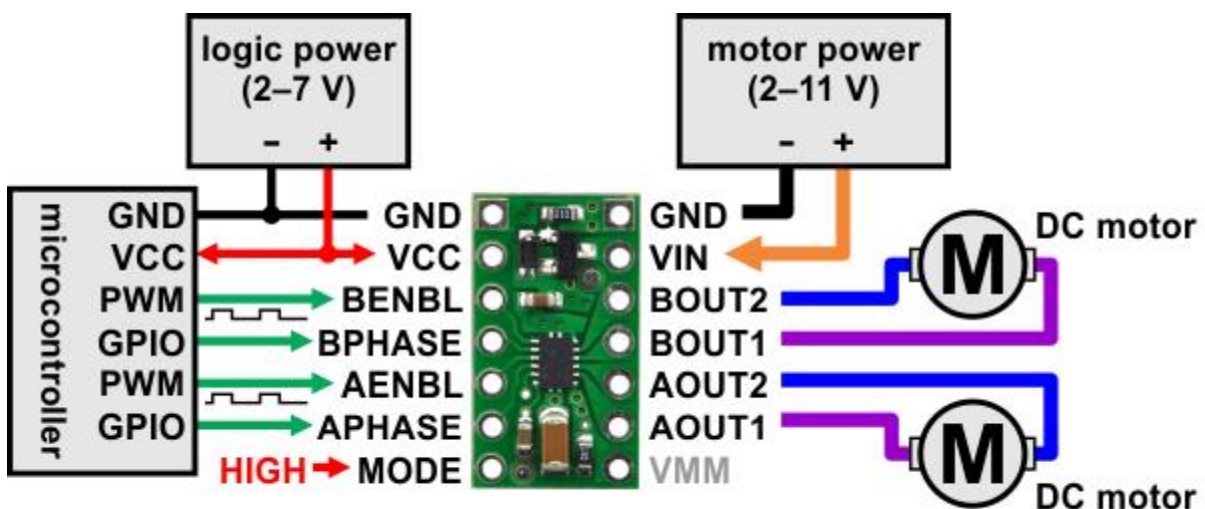
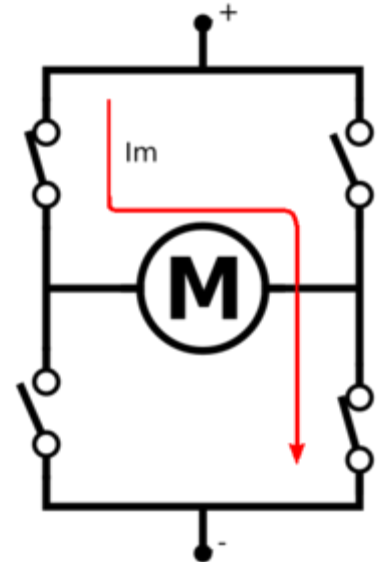
entrées #2 Calcul des fronts #3 Tant que

En rouge ce que j'ai rajouté au cas où ça a modifié ta mise en page :

Le pont en H est une structure électronique servant à contrôler la polarité aux bornes d'un dipôle.

Concernant la mise en place du montage au niveau hardware, nous avons utilisé le driver DRV8835 Polulu.

Le principe de fonctionnement de ce driver est décrit par le schéma suivant :



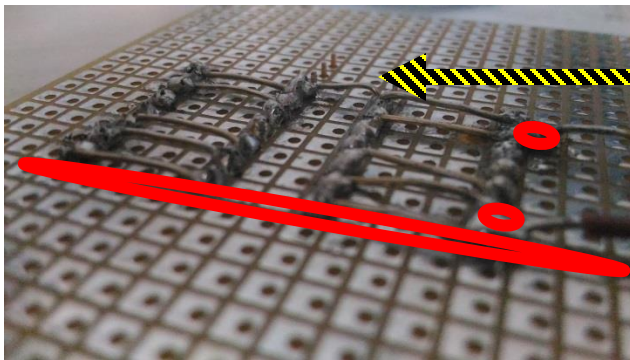
Pour le réaliser, nous avons commencé par fabriquer une piste sur lequel nous pourront le plugger et câbler les différentes entrées/sorties du montage.

Nous avons choisi de l'utiliser dans le mode = 1 car c'est celui qui est le plus adapté à mettre en œuvre pour notre application

Simplified drive/brake operation with MODE=1 (PHASE/ENABLE)				
xPHASE	xENABLE	xOUT1	xOUT2	operating mode
0	PWM	PWM	L	forward/brake at speed <i>PWM</i> %
1	PWM	L	PWM	reverse/brake at speed <i>PWM</i> %
X	0	L	L	brake low (outputs shorted to ground)

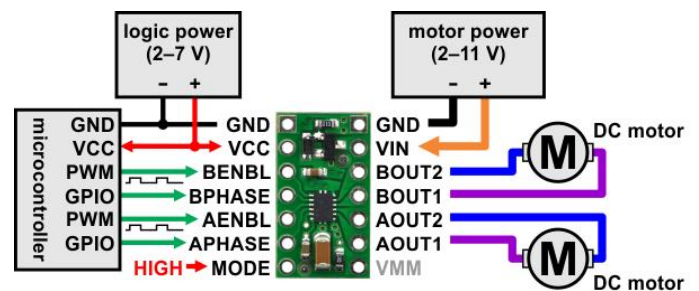
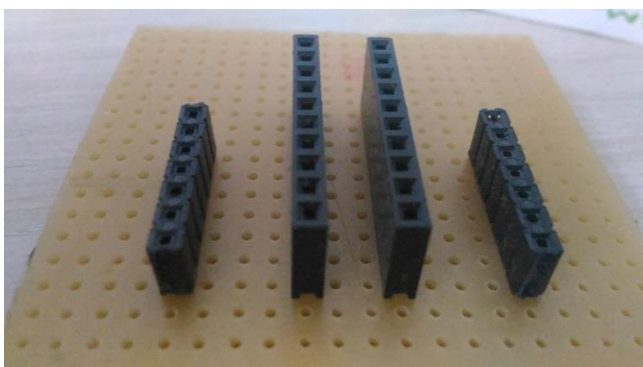
Avec ce montage nous pouvons inverser le sens de rotation de nos moteurs ainsi que définir la vitesse de rotation car l'alimentation de sortie est pilotée par un signal PWM avec lequel nous modulons la largeur d'impulsions.

Pour optimisé le routage nous avons reliés comme montré ci-après la pin MODE avec VCC (MODE=1) ainsi que les deux masses GND.



Ici le mode est forcé à 1 avec la mise en tension sur VCC.

Les autres pins servent à pluggier la PWM, la tension de fonctionnement du moteur ainsi que les sorties moteurs



Pour piloter cet ensemble, nous avons écrits des fonctions « haut niveau » nous permettant d'initialiser les PINS désirées ou imposées rapidement.

Une fonction d'initialisation pour chaque port prenant en entrée 16 paramètres correspondant aux 16 pins de chaque PORT.

Une autre fonction d'écriture fonctionnant sur le même principe.

Et enfin une fonction pour la PWM.

```
void initPatteGpioPortA(int a0, int a1, int a2, int a3, int a4, int a5, int a6, int a7, int a8, int a9, int a10, int a11, int a12, int a13, int a14, int a15);
void initPatteGpioPortB(int b0, int b1, int b2, int b3, int b4, int b5, int b6, int b7, int b8, int b9, int b10, int b11, int b12, int b13, int b14, int b15);
void initPatteGpioPortC(int c0, int c1, int c2, int c3, int c4, int c5, int c6, int c7, int c8, int c9, int c10, int c11, int c12, int c13, int c14, int c15);

void wPattePortA(int a0, int a1, int a2, int a3, int a4, int a5, int a6, int a7, int a8, int a9, int a10, int a11, int a12, int a13, int a14, int a15);
void wPattePortB(int b0, int b1, int b2, int b3, int b4, int b5, int b6, int b7, int b8, int b9, int b10, int b11, int b12, int b13, int b14, int b15);
void wPattePortC(int c0, int c1, int c2, int c3, int c4, int c5, int c6, int c7, int c8, int c9, int c10, int c11, int c12, int c13, int c14, int c15);

void Config_PA8_Mode_OutputPWM_TIM1_CH1(void);
```

Avec notre codage du Grafcet écrit en C nous avons pu piloter le système comme souhaité. Le capteur 3 permet de démarrer le moteur en actionnant un rapport cyclique de la PWM autour de 50%. Les capteurs 1 et 2 sont défini dans le cahier des charges comme des capteurs de positions, il permettent ici au taille crayon présent sur le tapis d'être detecté et pouvoir être déplacé selon un sens ou un autre grâce au pont H du driver.

